Advanced Robot Agents: Planning, Skills, Verification and Recovery

Course home · 简体中文 · PDF

Item	Details
Applicability	RAX, Gemini Robotics ER, SayCan, Code as Policies: conceptual comparison
Documentation review date	2026-10-18
License	CC BY 4.0 · Xiong Zeng and contributors

Verification scope: Research workflow and methodology discussion; no robot deployment validated. See version notes for pinned source references. Original teaching content uses CC BY 4.0; third-party quotations, figures and code retain their original licenses. Citation alone does not grant relicensing rights.

Contents

- 1. Responsibilities of Agents and Policies
- 2. Closed-Loop Workflow
  - 2.1 Executable Skill API
- 3. Representative Robot Agent Approaches
- 4. Planning, Verification and Belief State
- 5. Architectures and Safety
- 6. Shared Panda Experiment and Ablation
- 7. System Architecture: Constraining Robot-Directed Agent Actions
  - 7.1 Hierarchical Tools, Skills, and Task State
  - 7.2 A Closed Loop Requires Acceptance Tests, Not Just Feedback
  - 7.3 Different Time Scales Must Stay Separate
- 8. Detailed Contemporary Approaches
  - 8.1 RAX: Online Exploration, Compilation, and Held-Out Evaluation
  - 8.2 Gemini Robotics ER 2: Streaming Video and Tool Orchestration
  - 8.3 Comparing the Workflows
- 9. Independent Verification, Memory, and Risk
  - 9.1 Task Verification Beyond the Pose Estimate
  - 9.2 Choosing a Skill Under Benefits, Costs and Risks
  - 9.3 A Skill Library Must Preserve Evidence
- 10. Failure Recovery and Real-Robot Interfaces
  - 10.1 Diagnose Failures by Layer
  - 10.2 Shared Panda and Dexterous Tool Case

- 11. Evaluating Robot Agents Scientifically
  - 11.1 Minimum Ablation Matrix
  - 11.2 Metrics and Boundaries
- 12. Knowledge Boundaries

1. Responsibilities of Agents and Policies

A Robot Agent handles goal decomposition, skill selection, state updates, verification and replanning. VLA/imitative policies generate conditional robot actions; a motion controller handles timely execution. Write the hierarchy as $g \to z_k \to \pi_{\text{skill}}(a_t \mid o_t, z_k)$, where g is a task and z_k a parameterized skill. An agent must **not replace a high-rate servo loop**.

2. Closed-Loop Workflow

Starting from a language goal, an agent reads visual/proprioceptive state, proposes a reachable subgoal, calls a skill, then observes again and verifies success, recovery or stop conditions. Finishing a skill is not the same as satisfying the task. An occluded or unknown outcome cannot be marked successful.

2.1 Executable Skill API

Represent a skill as (name, preconditions, arguments, controller, timeout, success_predicate, fallback). A pick(object_id, grasp_pose) skill checks object identity, pose confidence and collision feasibility; on timeout it relocalizes or exits safely instead of retrying forever.

3. Representative Robot Agent Approaches

Gemini Robotics ER illustrates vision-language and spatial reasoning for task orchestration; RAX (Robot Agent Exploration) explores feedback-driven experience and skill reuse. Related ideas include SayCan for language-grounded skill affordances and Code as Policies for programmatic compositions. Capabilities depend on model version and robot integration: a paper demonstration does not imply calibration-free autonomous operation on arbitrary robots.

4. Planning, Verification and Belief State

For partial observability maintain a belief $b_t(s) = p(s_t \mid o_{\le t}, a_{<t})$. A grasp verifier such as $V_{\text{grasp}} = \mathbf{1}[\text{object lifted and retained}]$ should use visual and proprioceptive evidence over time. Low-confidence outcomes enter UNKNOWN and trigger more observation, rather than being labeled SUCCESS.

5. Architectures and Safety

Compare LLM/VLM plus fixed skills, VLM agents plus VLA, world-model candidate planning plus skills, and exploration agents plus experience memory. Predicted world-model imagery is not a ground-truth state measurement. Gate all high-level commands through workspace, collision,

force/velocity, latency, confidence and emergency-stop constraints. Low-level safety cannot depend on remote LLM response.

6. Shared Panda Experiment and Ablation

Shared Panda cube-on-table task: pick from randomized locations, place at goal, then recover from an externally displaced object. Compare A fixed script, B agent plus skills, C visual verification, D replanning, and E reusable memory while fixing the controller and evaluation distribution. Report success $\hat{p} = \frac{1}{N}\sum_i \mathbf{1}[\text{goal achieved}]$, latency, collisions, interventions, retries, recovery and tool/model costs.

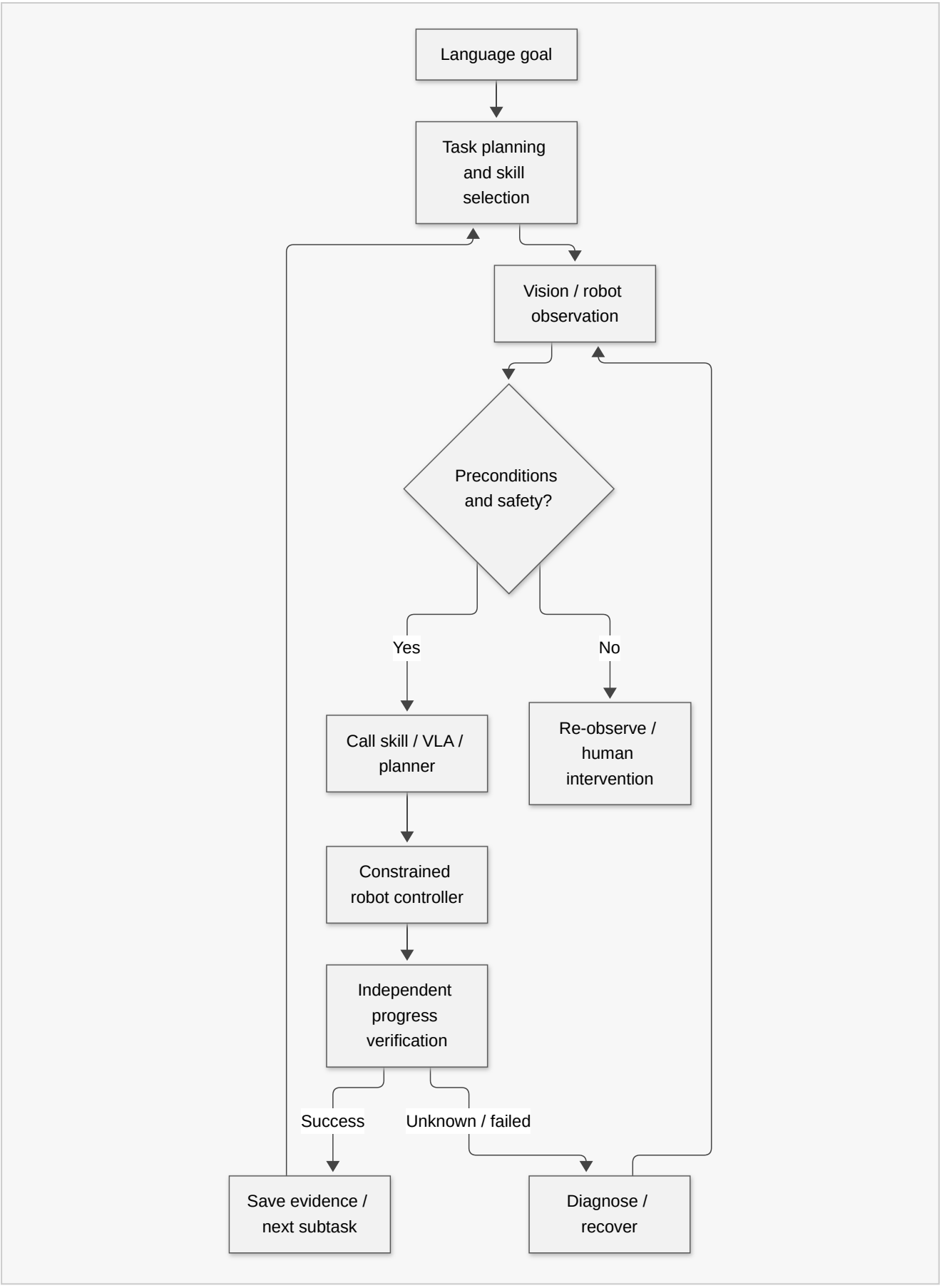
7. System Architecture: Constraining Robot-Directed Agent Actions

7.1 Hierarchical Tools, Skills, and Task State

Use structured task memory rather than an ever-growing free-form text log: object IDs, observation/pose timestamps, confidence, completed steps, gripper state, tool results, retry count and safety events. A skill $\omega = (I_\omega, \pi_\omega, \beta_\omega)$ has an initiation set I_ω, an internal policy π_ω and a termination rule β_ω; reliable deployment also needs a separate success verifier and recovery rule.

Useful tools include target search, pose query, navigation, gripper control, low-level VLA execution, progress verification and safe stop. The API contract must state coordinate frame, unit, parameter bounds, allowed control mode, timeout, observable completion, errors and audit logging. **The tool returning 'command accepted' is not proof the grasp succeeded.**

7.2 A Closed Loop Requires Acceptance Tests, Not Just Feedback



Separately check whether the command has terminated, the physical task outcome is achieved, and the next operation can begin safely. A language model saying 'done' is not a physical postcondition. Perception mathematics belongs in Visual State Estimation.

7.3 Different Time Scales Must Stay Separate

The top-level agent may be event-triggered; the estimator updates with camera observations; VLA policies may produce chunks; and the robot servo loop operates independently at higher frequency. Agent tokens must not directly drive servo commands. Bound each stage with explicit timeouts, stale-state detection and safe fallback. Select numerical rates from measured hardware timings, not universal assumptions.

8. Detailed Contemporary Approaches

8.1 RAX: Online Exploration, Compilation, and Held-Out Evaluation

RAX uses a general coding agent, including publicly documented Codex CLI or Claude Code setups, to manipulate through a robot tool API, collect observations and interaction feedback, remember useful preconditions, and compile successful explorations into a standalone robot controller program. The conceptual loop is **Explore → Compile → Evaluate**. Execution of the compiled program can avoid frequent calls to the original coding agent.

Published demonstrations cover different robots in simulation and hardware; the reported scaling results are tied to those evaluation tasks, not a universal real-robot success guarantee. The distinctive idea is episodic online exploration, persisted skill memory, compilation and evidence-backed testing; **RAX need not train a new VLA with gradients**.

8.2 Gemini Robotics ER 2: Streaming Video and Tool Orchestration

Gemini Robotics ER 2 is designed as a high-level embodied reasoner. It consumes video and language, plans steps, calls developer-defined robot APIs or lower-level VLA models, monitors progress, handles changes, and supports multi-robot coordination. It **does not normally replace the real-time motor control loop by emitting every joint torque itself**.

Continuous video enables decision-making during execution rather than once before a long open-loop episode. Progress classification and critical-event recognition help determine when to finish or correct steps; they still need physical postconditions and the independent hardware safety stack. The published Model Card should be used to check capabilities, limits, and supported interface versions.

8.3 Comparing the Workflows

Dimension	RAX	Gemini Robotics ER 2
Key mechanism	Explore robot API and compile learned skills	Streaming multimodal reasoning and tool selection
Adaptation	Trial-and-error, experience memory, verified code	Perceptual/linguistic generalization and feedback planning

Dimension	RAX	Gemini Robotics ER 2
Runtime	Compiled program may run without coding agent	Ongoing reasoning/stream processing may be used
Motor execution	Robot backend planners/controllers or learned policies	Attached VLA, navigation or robot tools
Primary challenges	Exploration safety, skill transfer, failed branches	Video mistakes, cost, latency, permission and safety

A possible hybrid uses RAX-style experience gathering to produce auditable skills and an ER-style reasoner to schedule them. This is a **proposed system design**, not a claim that the papers already established gains for such a hybrid.

9. Independent Verification, Memory, and Risk

9.1 Task Verification Beyond the Pose Estimate

Define an independent postcondition $V_\omega(o_{0:t}, q_{0:t}, g)$ for each skill. A grasp should verify stable object motion with the end effector, not just a closed gripper; insertion should check alignment, contact and final seating, not simply that the end effector reached a target. Verification states should distinguish SUCCESS, FAILURE and UNKNOWN. **UNKNOWN requires more sensing or safe human confirmation; it is not automatically SUCCESS.**

9.2 Choosing a Skill Under Benefits, Costs and Risks

A conceptual skill-choice objective (not an asserted internal optimization of RAX/ER 2) is

$ \omega_t^\star=\arg\max_{\omega\in\Omega(b_t)} \left[\mathbb E[R_{\rm goal}\mid b_t,\omega] - \lambda_c C(\omega)-\lambda_r {\rm Risk}(b_t,\omega) \right]. $

Only skills satisfying their preconditions and safety permissions enter $\Omega(b_t)$. The actuator service, not the language model, must enforce hard workspace and motor limits; this formula is not a hardware safety certificate.

9.3 A Skill Library Must Preserve Evidence

Store robot hardware, camera/URDF/controller versions, preconditions, execution traces, failure modes, postcondition checkers and unattended test count. The public RAX Skills Library illustrates retaining executable code with logs and success evidence. A skill that worked on another gripper or calibration revision must be revalidated before deployment.

10. Failure Recovery and Real-Robot Interfaces

10.1 Diagnose Failures by Layer

Failure	Evidence	Local recovery
Lost target	LOST state, invalid mask, stale image	Hold and reacquire

Failure	Evidence	Local recovery
Object slip	Relative pose change, gripper signal	Retreat safely and regrasp
Unreachable target	Failed IK / workspace limit	Change approach or skill
Tool timeout	No completion, excessive latency	Stop commands, inspect robot
Unknown progress	Conflicting verifier signals	Observe again or ask operator
Physical emergency stop	Hardware stop input	Stop immediately; no autonomous restart

Give the agent access only through a logged robot service. Preserve vendor safety controls, hard emergency stops and independent workspace/force/speed constraints. Uncertain observations must never justify arbitrary high-force exploratory movements.

10.2 Shared Panda and Dexterous Tool Case

In Panda + cube + table, preserve the same robot and controller while varying object starts, occlusions, displacement and grasp failures. The agent should localize → query trusted depth/pose → choose grasp skill → execute and verify → place → verify final state, recovering only affected subtasks. A screwdriver and dexterous-hand variant adds tool orientation, contact stability and reacquisition. Perception and policy training are taught in their own chapters; the agent chapter owns multi-step orchestration.

11. Evaluating Robot Agents Scientifically

11.1 Minimum Ablation Matrix

Compare A) fixed FSM/behavior tree, B) one-shot VLM planning, C) agent with execution feedback, D) independent success verifier added, E) failure replanning added, and F) persistent skill memory added. Additionally compare exploratory agent runs against compiled skill execution. Keep camera, robot API, controllers and held-out layouts fixed so improvements can be attributed to the orchestration layer.

11.2 Metrics and Boundaries

Report unseen-scene task success with confidence intervals, subtask completion, failure-detection recall, recovery success, tool-call count, interventions, inference/token cost, wall-clock completion time, controller latency and blocked unsafe actions. Exploration budgets and deployed inference costs must be reported separately. On real hardware, specify operator takeover conditions, maximum retries and permitted skill set.

12. Knowledge Boundaries

This chapter owns task-level orchestration, verification and recovery only. Consult Visual State Estimation for masks/depth/pose, Imitation Learning for collecting demonstrations and fitting BC/ACT/Flow policies, Flow Matching for full derivations, and GR00T/SONIC for system interfaces.

References: RAX · RAX code · skill library · Gemini Robotics ER 2 · SayCan · Code as Policies. No hardware benchmark or reproduction is claimed.